9 és ½. gyakorlat

Algoritmusok és adatszerkezetek 2.

Minimális összköltségű utak

Bellman-Ford algoritmus

Általános megoldandó feladat

Adott egy $G \in G_w$ gráf - irányított, súlyozott, amelyben negatív élsúlyok megengedettek, de nincs benne negatív kör. Adjunk algoritmust, amely meghatározza egy s start csúcsból az összes többi v csúcsba vezető legalacsonyabb összköltségű utat. Az algoritmus eredménye a szokásos d[] távolság és π[] leszármazottsági tömbök.

1. Feladat

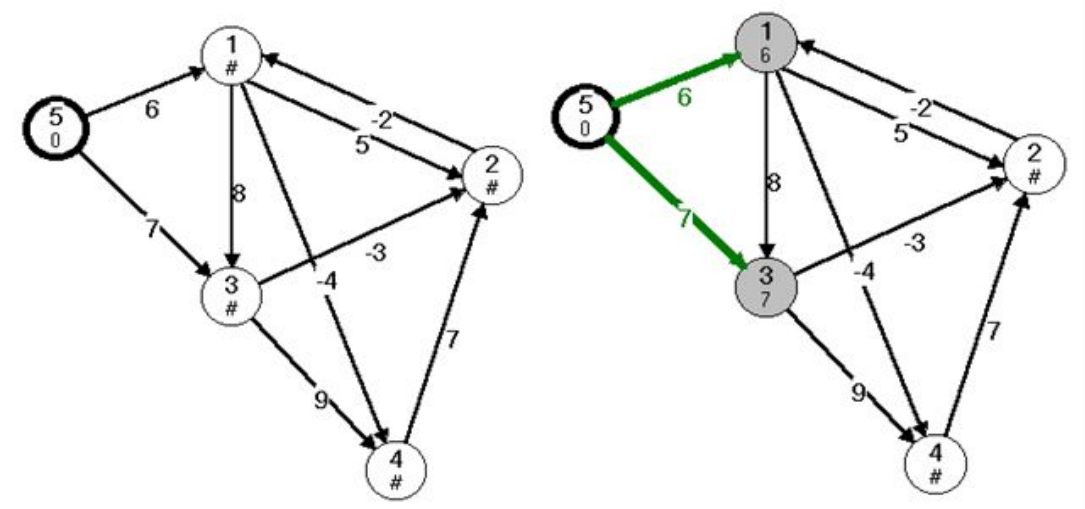
Adott egy $G \in G_w$ gráf - irányított, súlyozott, amelyben negatív élsúlyok megengedettek, de nincs benne negatív kör. Adjunk algoritmust, amely meghatározza egy s start csúcsból az összes többi v csúcsba vezető legfeljebb 1 hosszú (1 élből álló) legalacsonyabb összköltségű utat.

Az algoritmus eredménye a szokásos $d_1[]$ távolság és $\pi_1[]$ leszármazottsági tömbök.

Ötlet

Menjünk végig az összes s-ből kivezető (s,v) élen és nézzük meg, hogy annak segítségével az egyes csúcsok milyen költségen érhetőek el és ezt állítsuk be v-hez. Minden ilyen esetben legyen a $\pi_1[v]=s$.

Szemléltetés



2. Feladat

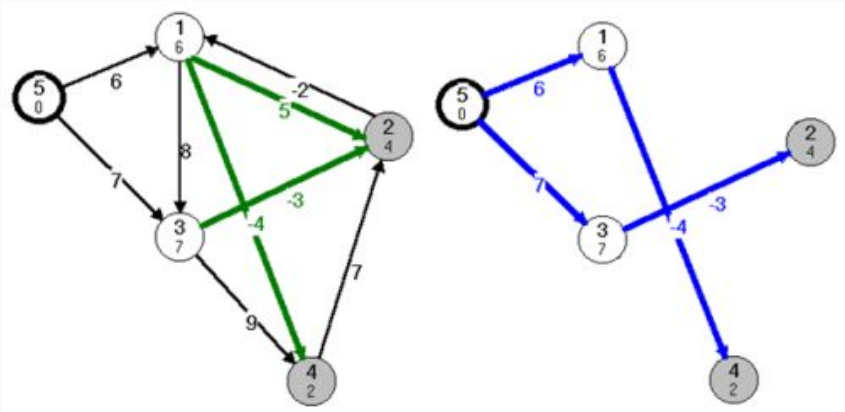
Adott egy $G \in G_w$ gráf - irányított, súlyozott, amelyben negatív élsúlyok megengedettek, de nincs benne negatív kör. Adjunk algoritmust, amely meghatározza egy s start csúcsból az összes többi v csúcsba vezető legfeljebb 2 hosszú (2 élből álló) legalacsonyabb összköltségű utat, feltéve, hogy az 1 hosszú legrövidebb utakat ismerjük.

Az algoritmus eredménye a szokásos $d_2[]$ távolság és $\pi_2[]$ leszármazottsági tömbök, az 1 hosszú utak pedig a $d_1[]$ távolság és $\pi_1[]$ leszármazottsági tömbök segítségével adottak.

Ötlet

Próbáljunk oda minden lehetséges (u,v) élt az u-ba vezető 1 hosszú utak végére. Ha bármelyik esetben a $d_1[u]+w(u,v) < d_1[v]$, akkor $d_2[v]:=d_1[u]+w(u,v)$ és $\pi_2[v] := u$.

Szemléltetés



3. Feladat

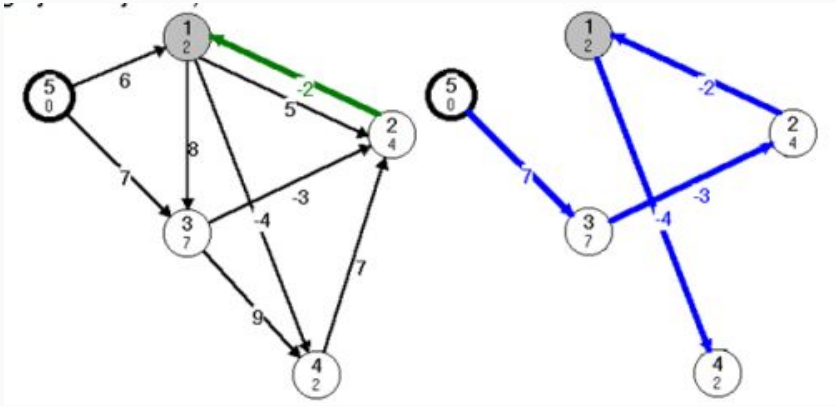
Adott egy $G \in G_w$ gráf - irányított, súlyozott, amelyben negatív élsúlyok megengedettek, de nincs benne negatív kör. Adjunk algoritmust, amely meghatározza egy s start csúcsból az összes többi v csúcsba vezető legfeljebb k hosszú (k élből álló) legalacsonyabb összköltségű utat, feltéve, hogy az k-1 hosszú legrövidebb utakat ismerjük.

Az algoritmus eredménye a szokásos $d_k[]$ távolság és $\pi_k[]$ leszármazottsági tömbök, az 1 hosszú utak pedig a $d_{k-1}[]$ távolság és $\pi_{k-1}[]$ leszármazottsági tömbök segítségével adottak.

Ötlet

Próbáljunk oda minden lehetséges (u,v) élt az u-ba vezető k-1 hosszú utak végére. Ha bármelyik esetben a $d_{k-1}[u]+w(u,v) < d_{k-1}[v]$, akkor $d_k[v]:=d_{k-1}[u]+w(u,v)$ és $\pi_k[v] := u$.

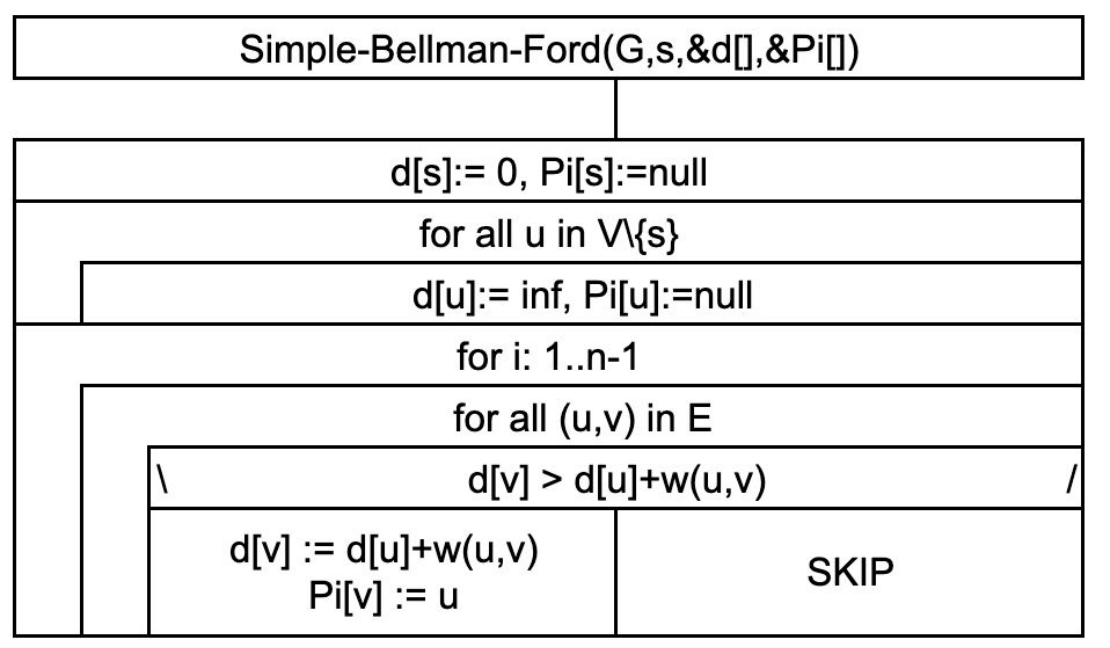
Szemléltetés



4. Feladat

Adott egy $G \in G_w$ gráf - irányított, súlyozott, amelyben negatív élsúlyok megengedettek, de nincs benne negatív kör. Adjunk algoritmust, amely meghatározza egy s start csúcsból az összes többi v csúcsba vezető legalacsonyabb összköltségű utat. Az algoritmus eredménye a szokásos d[] távolság és π[] leszármazottsági tömbök.

Simple Bellmann-Ford



5. Feladat

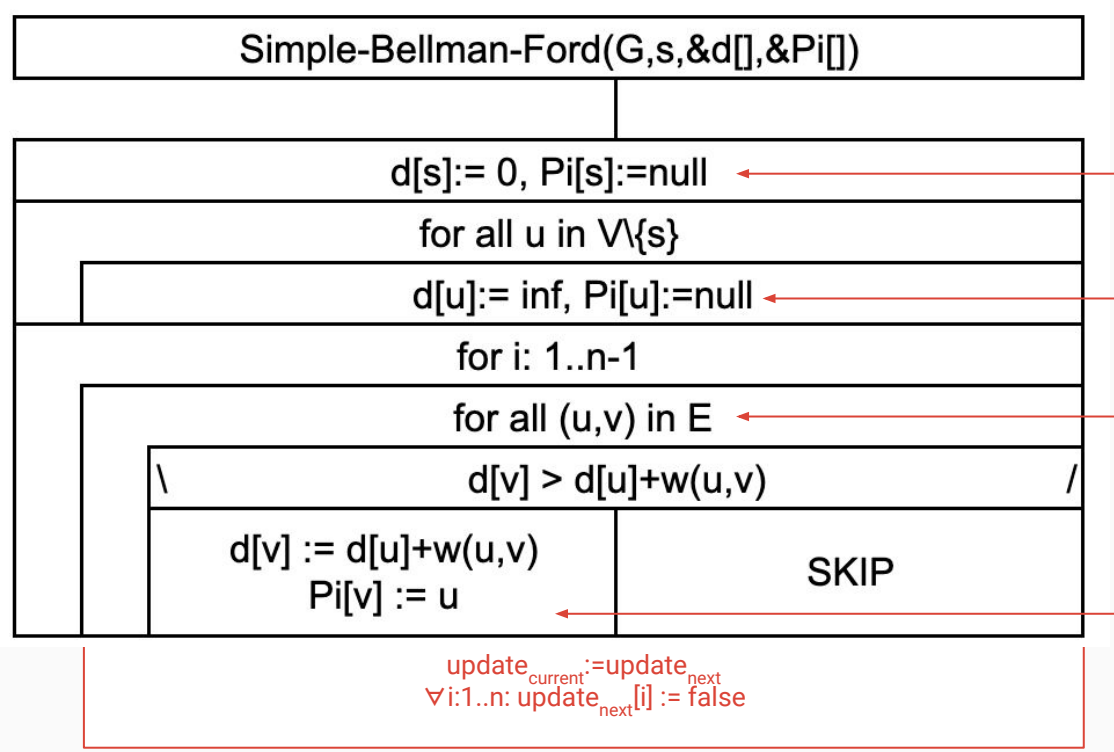
Gondoljuk át, hogy hogyan tudjuk a vizsgált élek számát csökkenteni az előbbiekben kitalált algoritmus egyes iterációiban.

Ötlet

Gondoljuk át: a k-adik lépésben a k-1 hosszú utak végére “odapróbáljuk” az összes élt. Tehát egy tetszőleges u-ból kimenő összes (u,v) élt megvizsgáltunk. Ha az adott iterációban a d[u] nem változik (tehát nincs k db élből álló olcsóbb út, mint k-1 élből álló), akkor viszont k. iterációban ugyanazokat az éleket próbálnánk a végére, amit a k. Iterációban.

Tehát ha a k. iterációban a d[u] nem változik, akkor a következőben nem kell vizsgálni. Hol kell változtatni az algoritmuson, hogy ezt kihasználjuk?

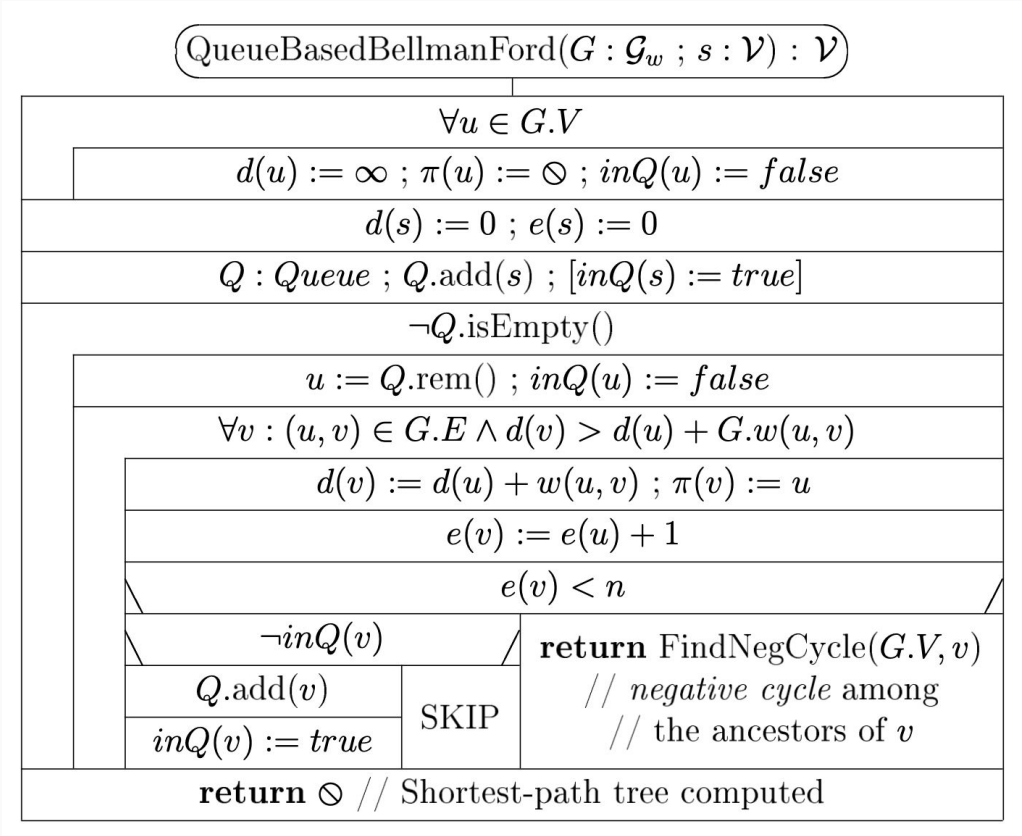
Bellmann-Ford javítása



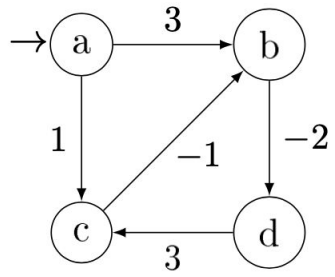
Jobb ötlet

Használjunk sort, kezdetben legyen benne az s. Mindig vegyünk ki egy csúcsot a sorból és tegyük annak végére azokat a csúcsokat, amelyek változtak.

Sor alapú Bellman-Ford algoritmus



Szemléltetés

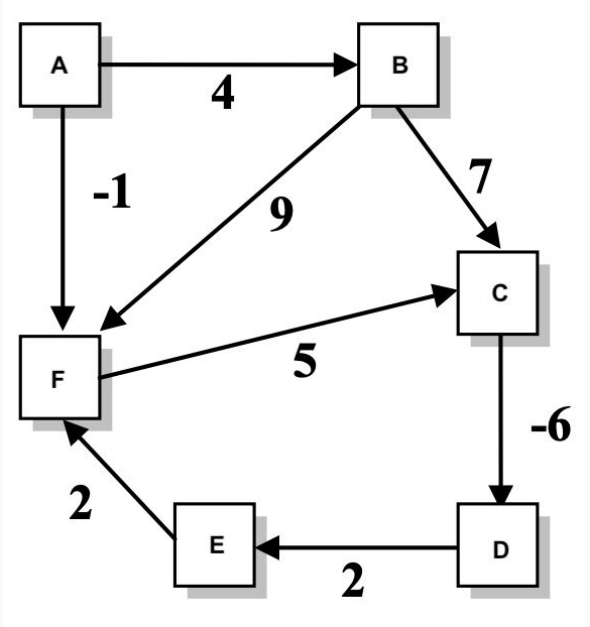


a → b, 3 ; c, 1.
b → d, −2.
c → b, −1.
d → c, 3.

$d; e$ változásai				kiter-	Q :	π változásai				Menet
a	b	c	d	jesztett	Queue	a	b	c	d	
0; 0	∞	∞	∞	csúcs: $d; e$	⟨ a ⟩	⦸	⦸	⦸	⦸	
	3; 1	1; 1		a: 0; 0	⟨ b, c ⟩		a	a		0.
			1; 2	b: 3; 1	⟨ c, d ⟩				b	1.
	0; 2			c: 1; 1	⟨ d, b ⟩		c			1.
				d: 1; 2	⟨ b ⟩					2.
			−2; 3	b: 0; 2	⟨ d ⟩				b	2.
				d: −2; 3	⟨ ⟩					3.
0	0	1	−2	végső d és π értékek		⦸	c	a	b	-

6. Feladat

Szemléltessük a Bellman–Ford-algoritmus működését az alábbi gráfon.



Megoldás

lépés	d[], e[] változásai								Pi[] változásai					
	A	B	C	D	E	F	Kiterjesztett csúcs	Q:Queue	A	B	C	D	E	F
0	0,0	inf	inf	inf	inf	inf	-	<A>	/	/	/	/	/	/
1		4,1				-1,1	A:0,0	<F,B>		A				A
2			13,2				B:4,1	<C,F>			B			
3			4,2				F:-1,1	<C>			F			
4				-2,3			C:4,2	<D>				C		
5					0,4		D:-2,3	<E>					D	
6							E:0,4	<>						

7. Feladat

Arbitrázs a valutaváltási árfolyamokban rejlő egyenlőtlenségek olyan hasznosítása, amikor egy valuta 1 egységéből elindulva, egy valutaváltási sorozat lefolytatása után ugyanazon valuta 1 egységénél nagyobb értékére teszünk szert. (Pl.: 1 dollárért veszünk 0,7 fontot, majd a fontot átváltjuk frankra 9,5-es szorzóval, majd a frankot ismét dollárra váltjuk 0,16-os szorzóval, ekkor 0,7*9,5*0,16=1,064. Tehát 6,4%-os haszonra tettünk szert.)

Adjunk algoritmust, amely meghatározza, hogy létezik-e a valutáknak ilyen sorozata, és ha létezik, adjunk meg egy ilyet.

Megoldás:

Fogalmazzuk át:

Adott egy olyan irányított gráf, amelyben a csúcsok az egyes valuták, és az élek súlyai az árfolyamok. Ebben egy olyan $u_0, u_1, u_2 \dots u_k$ kört keresünk ($u_0=u_k$), hogy:

$\Pi_{i=0..k-1} w(u_i,u_{i+1}) > 1$ //log függvény alkalmazása mindkét oldalon

$log(\Pi_{i=0..k-1} w(u_i,u_{i+1})) > log(1)$ //log(a*b) = log(a)+log(b) egyenlőséget:

$\Sigma_{i=0..k-1} log(w(u_i,u_{i+1})) > 0$ //*(-1)

$\Sigma_{i=0..k-1} -log(w(u_i,u_{i+1})) < 0$ //→ negatív összköltségű kört keresünk → Bellman-Ford